

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

12. August 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Enrico Ebert) . . . 1
1.2 Zielsetzung der Arbeit (Duy Tan Pham) . . . 1
1.3 Methodik (Markus Nöth) . . . 1

2 Übergreifende Architektur & Systemdesign **1**
2.1 Technologiestack (Richard Schröder) . . . 1
2.2 Modulare Softwarearchitektur (Markus Nöth) . . . 2
2.3 Unified UI-Konzept (Duy Tan Pham) . . . 2
2.4 Simulator-Architektur (Richard Schröder) . . . 2
2.5 Datenmodellierung (Richard Schröder) . . . 2

3 Einsatz von Künstlicher Intelligenz im Projekt **2**
3.1 KI Reflexion (Markus Nöth) . . . 2
3.1.1 KI als Co-Developer (Markus Nöth) . . . 2
3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth) . . . 2
3.1.3 Kritische Reflexion (Markus Nöth) . . . 2
3.2 KI Reflexion (Richard Schröder) . . . 2
3.2.1 KI als Co-Developer (Richard Schröder) . . . 2
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 3
3.2.3 Kritische Reflexion (Richard Schröder) . . . 4
3.3 KI Reflexion (Enrico Ebert) . . . 4
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 4
3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert) . . . 4
3.3.3 Kritische Reflexion (Enrico Ebert) . . . 4
3.4 KI Reflexion (Duy Tan Pham) . . . 4
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 5
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 5
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 5
3.5 KI Reflexion (Kristian Popov) . . . 6
3.5.1 KI als Co-Developer (Kristian Popov) . . . 6
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 7
3.5.3 Kritische Reflexion (Kristian Popov) . . . 8

4 Designentscheidungen der Fachmodule **8**
4.1 Turing Maschine (Richard Schröder) . . . 8
4.1.1 Entscheidungen in der Architektur (Richard Schröder) . . . 8
4.1.2 Entscheidungen im UI (Richard Schröder) . . . 10
4.1.3 Andere Entscheidungen (Richard Schröder) . . . 11
4.2 Endliche Automaten (Kristian Popov) . . . 12
4.2.1 Entscheidungen in der Architektur (Kristian Popov) . . . 12
4.2.2 Entscheidungen im UI (Kristian Popov) . . . 13
4.2.3 Andere Entscheidungen (Kristian Popov) . . . 13
4.3 Reguläre Sprachen (Kristian Popov) . . . 15
4.3.1 Entscheidungen in der Architektur (Kristian Popov) . . . 15
4.3.2 Entscheidungen im UI (Kristian Popov) . . . 16
4.3.3 Andere Entscheidungen (Kristian Popov) . . . 17
4.4 Zahlensysteme (Enrico Ebert) . . . 17
4.4.1 Entscheidungen in der Architektur (Enrico Ebert) . . . 17

4.4.2 Entscheidungen im UI (Enrico Ebert) . . . 17
4.4.3 Andere Entscheidungen (Enrico Ebert) . . . 17
4.5 Moore & Mealy Automaten (Markus Nöth) . . . 17
4.5.1 Entscheidungen in der Architektur (Markus Nöth) . . . 17
4.5.2 Entscheidungen im UI (Markus Nöth) . . . 17
4.5.3 Andere Entscheidungen (Markus Nöth) . . . 17
4.6 Disjunktive Normalform (Duy Tan Pham) . . . 17
4.6.1 Entscheidungen in der Architektur (Duy Tan Pham) . . . 18
4.6.2 Entscheidungen im UI (Duy Tan Pham) . . . 19
4.6.3 Andere Entscheidungen (Duy Tan Pham) . . . 19
4.7 Encoding (Enrico Ebert) . . . 20
4.7.1 Entscheidungen in der Architektur (Enrico Ebert) . . . 20
4.7.2 Entscheidungen im UI (Enrico Ebert) . . . 20
4.7.3 Andere Entscheidungen (Enrico Ebert) . . . 20
4.8 Java (Markus Nöth) . . . 20
4.8.1 Entscheidungen in der Architektur (Markus Nöth) . . . 20
4.8.2 Entscheidungen im UI (Markus Nöth) . . . 20
4.8.3 Andere Entscheidungen (Markus Nöth) . . . 20

5 Evaluation & Projekterfahrung 20
5.1 Qualitätssicherung (Enrico Ebert) . . . 20
5.2 Abgleich mit dem Lastenheft (Kristian Popov) . . . 20
5.3 Lessons Learned (Alle) . . . 20
5.4 Design-Iterationen (Duy Tan Pham) . . . 20

6 Fazit & Ausblick 20
6.1 Zusammenfassung der Projektergebnisse (Alle) . . . 20
6.2 Perspektiven (Kristian Popov) . . . 20

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . i

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Eine Arbeit von Technische Hochschule Würzburg-Schweinfurt (THWS) Studenten.

1.1 Problemstellung (Enrico Ebert)

Die Hürde der Abstraktion in Grundlagen Informatik

1.2 Zielsetzung der Arbeit (Duy Tan Pham)

Didaktische Aufbereitung komplexer Modelle durch interaktive Simulationen

1.3 Methodik (Markus Nöth)

Vorgehensweise im Team und Rollenverteilung

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Richard Schröder)

Die Anwendung wurde als eine Client-only Webanwendung umgesetzt, d. h. es gibt kein Backend, das Funktionen ausführt außer dem bereitstellen der Seite selbst. Deshalb wurden nur Frontend Technologien verwendet. Den Kern der Seite bildet `React 19` mit dem `React Router`, welcher zusätzlich Lazy Loading für bessere Performance ermöglicht. Die Logik selbst wurde in `TypeScript` entwickelt. Der lokale Entwicklungsserver wurde mit `Vite` betrieben, was auch einen optimierten und statischen Build der Seite für das spätere Deployment generiert. Außerdem werden über `vitest` die Unittests abgebildet.

Das Styling der Seite basiert auf `Tailwind CSS 4` und den damit erstellten Utility-Klassen der einzelnen React Komponenten sowie diversen CSS-Variablen um z. B. Farben für die ganze Anwendung einheitlich zu halten. Mit `shadcn/ui` wurde die grundlegende Designbasis erstellt und `Tailwind CSS` um einige Komponenten und Theme-Strukturen ergänzt. Die Icons wurden per `Lucide React` direkt als React-Komponenten zur Verfügung gestellt.

Für einige globale Benutzeroberflächenzustände kommt `Zustand` zum Einsatz. Damit werden die Einstellung der Seitenleiste sowie die gewählte Darstellung zentral verwaltet, zusätzlich wird die Theme-Einstellung im Local Storage gespeichert, sodass sie nach einem erneuten Aufruf der Anwendung erhalten bleibt. Fachliche Zustände der einzelnen Simulatoren werden hingegen überwiegend lokal in den jeweiligen Komponenten und zugehörigen Domänenmodellen geführt.

Für die Internationalisierung wurde `i18next` bzw. `react-i18next` eingesetzt, somit kann die gesamte Anwendung einfach in verschiedene Sprachen übersetzt werden.

Im Java Bereich wurde zudem `monaco` als Editor eingebunden, der Java Code wird anschließend mit der `TeaVM` direkt im Browser kompiliert und ausgeführt.

Die Versionsverwaltung erfolgt mit Git, das zentrale Repository und die automatisierten Abläufe werden über das THWS GitLab organisiert. GitLab CI führt unter anderem Formatierungs- und Lint-Prüfungen, Unit-Tests, den Build sowie End-to-End-Tests aus. Die konkrete Umsetzung der CI/CD-Pipeline und des GitLab Runners wird in Kapitel 5.1 behandelt.

2.2 Modulare Softwarearchitektur (Markus Nöth)

Single Source of Truth (`moduleCatalog.tsx`), lazy Loading, Shared Topic Pattern – Sicherstellung der Erweiterbarkeit für neue Themen

2.3 Unified UI-Konzept (Duy Tan Pham)

Wiederverwendbare Shared Components, konsistentes Designsystem (CSS-Variablen, Dark Mode, moduleigene Akzentfarben, Geist Font)

2.4 Simulator-Architektur (Richard Schröder)

Das `ISimulator<TConfig, TState>`-Interface und der generische `useSimulationEngine`-Hook als Grundlage aller schrittweisen Simulationen

2.5 Datenmodellierung (Richard Schröder)

Import/Export über `ISimulatorTransferCodec` mit Base64-Kodierung für Vorlesungskompatibilität

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI Reflexion (Markus Nöth)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.1.1 KI als Co-Developer (Markus Nöth)

Reflektieren und so...

3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth)

Reflektieren und so...

3.1.3 Kritische Reflexion (Markus Nöth)

Reflektieren und so...

3.2 KI Reflexion (Richard Schröder)

3.2.1 KI als Co-Developer (Richard Schröder)

Im Rahmen der Projektarbeit wurden verschiedene KI-gestützte Werkzeuge als unterstützende Systeme in der Softwareentwicklung eingesetzt. Dazu zählten insbesondere ChatGPT, Codex in den Entwicklungsumgebungen WebStorm und Zed sowie lokal ausgeführte Modelle über Ollama in Zed. Ergänzend wurde Gemini vereinzelt für Rechercheaufgaben genutzt, wenn zusätzliche externe Informationen oder alternative Formulierungen benötigt wurden.

Der Einsatz dieser Werkzeuge hat den Entwicklungsprozess deutlich erleichtert, da Entwürfe, Implementierungsvorschläge und Reviews wesentlich schneller erstellt werden konnten. Gleichzeitig zeigte sich jedoch, dass die generierten Ergebnisse nicht unkritisch übernommen werden dürfen.

Eine fachliche Kontrolle blieb in allen Phasen notwendig, insbesondere bei Architekturentscheidungen, bei Änderungen an bestehendem Code und bei der Bewertung der Testabdeckung.

Das allgemeine Vorgehen beim Einsatz von KI als Co-Developer bestand aus mehreren wiederkehrenden Schritten:

1. Zunächst wurde der gewünschte Zielzustand des Systems möglichst detailliert beschrieben. Auf dieser Grundlage wurde ein erster Architekturvorschlag durch das KI-System erzeugt, der als Ausgangspunkt für die weitere Planung diente.
2. Anschließend wurde der Architekturvorschlag fachlich geprüft und überarbeitet. Dies war erforderlich, da die ersten Vorschläge teilweise zu allgemein waren oder sich nicht optimal in die bereits bestehende Systemarchitektur einfügten.
3. Der verbesserte Architekturentwurf wurde danach in konkrete Arbeitspakete zerlegt. Auch diese Aufteilung wurde kontrolliert, um sicherzustellen, dass die Aufgaben fachlich sinnvoll abgegrenzt waren und keine relevanten Abhängigkeiten übersehen wurden.
4. Die einzelnen Arbeitspakete wurden anschließend in separaten Prompts bearbeitet, wobei die Gesamtarchitektur weiterhin als Kontext mitgegeben wurde. Nach jeder Umsetzung wurde die generierte Implementierung manuell überprüft, um Fehler, unpassende Änderungen oder Abweichungen vom geplanten Vorgehen frühzeitig zu erkennen.
5. Zusätzlich wurde der generierte Code in einem neuen Chat erneut durch KI geprüft. Dabei lag der Fokus auf der Übereinstimmung mit dem zuvor definierten Architekturdokument sowie auf einem allgemeinen Code Review hinsichtlich Verständlichkeit, Struktur und möglicher Schwachstellen.
6. Besondere Aufmerksamkeit wurde auf korrekte und vollständige Tests gelegt. Neben Unit-Tests wurden auch End-to-End-Tests betrachtet, da diese für die Absicherung der implementierten Nutzerabläufe und der Integration verschiedener Komponenten besonders relevant waren.

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Ein weiteres Einsatzfeld der KI-Systeme war die automatisierte Generierung von Aufgaben. Zu Beginn waren die Ergebnisse nur eingeschränkt verwendbar, insbesondere bei Aufgaben zur Turingmaschine. Häufig wurde das geforderte JSON Format nicht korrekt eingehalten, und die erzeugten Lösungen zeigten, dass ChatGPT und Codex in diesem speziellen Themenbereich nicht ohne zusätzliche Vorgaben als zuverlässige Experten eingesetzt werden konnten.

Die Qualität der Ergebnisse verbesserte sich, nachdem mehrere korrekt aufgebaute Turingmaschinen erstellt und als Kontext für weitere Generierungen verwendet wurden. Dadurch konnten sich die Modelle besser an das gewünschte Format und die erwartete Aufgabenstruktur orientieren. Dennoch blieb weiterhin eine fachliche Kontrolle notwendig, da auch mit verbessertem Kontext nicht alle generierten Turingmaschinen vollständig korrekt waren.

Das Nachbauen von Aufgaben aus Skript und Übungsblättern war hingegen gut umsetzbar, wenn ausreichend JSON Beispiele als Kontext bereitgestellt wurden. Besonders hilfreich war dabei die Kombination aus bestehenden Aufgabenbeispielen und einem Screenshot der jeweiligen Lösung. Auf dieser Grundlage konnte ChatGPT vergleichsweise zuverlässig Lösungen erzeugen, die sich an Struktur und Formulierung der Vorlagen orientierten.

3.2.3 Kritische Reflexion (Richard Schröder)

Insgesamt stellte der Einsatz von KI im Projekt eine erhebliche Unterstützung dar, insbesondere um Entwicklungsaufgaben schneller zu strukturieren, Implementierungen vorzubereiten und alternative Lösungsansätze zu prüfen. Dadurch konnte effizienter gearbeitet werden, ohne jeden Entwurf oder jede technische Detailentscheidung vollständig von Grund auf manuell ausarbeiten zu müssen. Der Nutzen war jedoch besonders dann hoch, wenn die Anfragen klar eingegrenzt und die Ergebnisse anschließend überprüft wurden.

Eine zentrale Einschränkung bestand darin, dass zu große Aufgaben nicht zuverlässig in einem einzigen Schritt bearbeitet werden konnten. Wurde beispielsweise versucht, eine vollständige Turingmaschine mit allen Details auf einmal generieren zu lassen, konnten Teile der Anforderungen aus dem Kontext verloren gehen oder fehlerhaft umgesetzt werden. Daher erwies es sich als sinnvoller, komplexe Aufgaben in kleinere Einheiten zu zerlegen und diese nacheinander zu bearbeiten.

Darüber hinaus zeigte sich gelegentlich eine Tendenz der KI-Systeme, einfache Problemstellungen unnötig komplex zu lösen. In solchen Fällen wurden umfangreiche Strukturen, zusätzliche Abstraktionen oder technische Umwege vorgeschlagen, obwohl eine deutlich einfachere Lösung ausreichend gewesen wäre. Dies machte eine kritische Bewertung der vorgeschlagenen Architektur und Implementierung besonders wichtig.

Problematisch war außerdem, dass KI-generierte Änderungen nicht immer an der fachlich passenden Stelle vorgenommen wurden. Teilweise wurden allgemeine oder gemeinsam genutzte Komponenten angepasst, obwohl eine lokale Änderung innerhalb des jeweiligen Moduls angemessener gewesen wäre. Solche Änderungen können unerwünschte Auswirkungen auf andere Teile des Systems haben und verdeutlichen, dass KI-Unterstützung nur in Verbindung mit einem guten Verständnis der bestehenden Codebasis sinnvoll eingesetzt werden sollte.

3.3 KI Reflexion (Enrico Ebert)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.3.1 KI als Co-Developer (Enrico Ebert)

Reflektieren und so...

3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert)

Reflektieren und so...

3.3.3 Kritische Reflexion (Enrico Ebert)

Reflektieren und so...

3.4 KI Reflexion (Duy Tan Pham)

Für meinen Teil, die Disjunktive Normalform, habe ich durchgehend mit KI-Werkzeugen gearbeitet. Codex war das Hauptwerkzeug beim Entwickeln und direkt in VS Code eingebunden. Claude habe ich für Analyse, Refactorings sowie für Kontext- und Wissensdokumente genutzt. Die KI diente dabei nicht nur als Entwicklungswerkzeug, sondern auch zur Aufarbeitung der fachlichen Grundlagen. Einzelne Themen wie die Funktionsweise der DNF, das Verfahren von

Quine-McCluskey und die Karnaugh-Diagramme habe ich mir von der KI erklären lassen. Dadurch wurde nicht nur das Schreiben des Codes beschleunigt, sondern auch das Verständnis des Stoffs unterstützt.

3.4.1 KI als Co-Developer (Duy Tan Pham)

Meine Arbeitsweise mit der KI war vor allem dialogisch und nicht befehlend. Statt fertige Vorgaben zu machen, habe ich die KI meist zuerst nach einer Einschätzung und einem Lösungsansatz gefragt, bevor etwas umgesetzt wurde. Die eigentliche Entscheidung ist dann schrittweise im Austausch entstanden. Dieses Nachfragen war zugleich meine Kontrolle. Weil ich die Vorschläge hinterfragt habe, bevor ich sie übernahm, blieb die fachliche Entscheidung bei mir, und Fehler fielen früher auf.

Dabei haben sich einige feste Muster herausgebildet. Grundsätzlich stand der Lösungsansatz vor der Umsetzung. Die KI sollte einen Vorschlag zuerst erklären, bevor sie Code erzeugt. Ergänzend habe ich sie ausdrücklich um eine ehrliche Einschätzung gebeten, etwa zur Benennung von Funktionen oder zu möglichen unsauberen Stellen und Sonderfällen. Die Oberfläche habe ich häufig anhand von Screenshots beurteilen und schrittweise verbessern lassen, bis sie sauber und einheitlich war. Einheitlichkeit war dabei eine wichtige Leitplanke. Ich habe den DNF-Bereich an den gemeinsamen Gestaltungs- und Interaktionsmustern der Lernplattform ausgerichtet. Zum Abschluss gehörten gezielte Tests, Linting, Build, Formatierung mit Prettier und punktuell das Analysewerkzeug `fallow` als zusätzliche Prüfung.

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Für den DNF-Bereich habe ich die Übungsaufgaben nicht von einer KI erzeugen lassen. Die Aufgaben sind als feste Übungsdefinitionen im Code hinterlegt und verwenden Ausdrücke oder Wahrheitstabellen mit höchstens vier Variablen. Die erwarteten Ergebnisse werden mit der DNF-Fachlogik bestimmt. Dadurch bleiben die Aufgaben reproduzierbar und benötigen für die Prüfung keine separat gepflegten Lösungstexte. Die KI hat hier bei der Umsetzung der Logik und bei einzelnen Beispielausdrücken unterstützt, nicht bei der Auswahl der Aufgaben.

3.4.3 Kritische Reflexion (Duy Tan Pham)

Die wichtigste Erkenntnis war für mich, dass man sich bei deterministischer Informatik-Logik nicht blind auf die KI verlassen kann. Ein Beispiel ist die Minimierung. Die damalige Lösung hat die Restüberdeckung greedy gewählt. Erst meine konkrete Nachfrage, ob damit die literalminimale DNF übersehen werden kann, führte zur Diagnose und anschließend zur exakten Lösung. Ich musste das Korrektheitskriterium also selbst benennen und am Code prüfen. Die KI stellte dieses Korrektheitskriterium nicht eigenständig sicher.

Ein zweites Thema war das begrenzte Kontextfenster. In langen Chats ging Kontext verloren, deshalb habe ich mir Zusammenfassungen für neue Chats erstellen lassen und wichtige Fakten in eigene Wissensdokumente ausgelagert. Um das Repository herum entstand dadurch ein lokaler Arbeitsbereich mit Vorlesungsunterlagen, Übungsblättern, Lastenheft, Work-Item-Exporten, Videoquellen und ausgewählten Chatprotokollen. Kurze Handoff-Dateien und aufgabenspezifische Skills steuerten, welche Quellen ein neuer Agent für eine Aufgabe lesen und welche Prüfungen er durchführen sollte. Dadurch musste nicht jeder Chat den gesamten Projektverlauf neu einlesen. Die zusätzlichen Dateien waren dabei nur Orientierung und Prozessbelege. Technische Aussagen wurden weiterhin am aktuellen Code, an Tests oder an Commits geprüft.

Außerdem musste ich auf den Umfang der Anfragen achten. Zu große Aufgaben auf einmal wurden unzuverlässig, kleinere Schritte lieferten bessere Ergebnisse. Auch eine Tendenz zu

unnötig komplexen Lösungen war ein wiederkehrendes Muster. Solche Vorschläge habe ich bewusst wieder zurückgenommen, etwa einen unnötigen Refactor der Eingabe-Komponenten.

Ein wiederkehrendes Risiko waren unbelegte technische Annahmen. Aussagen der KI zu externen Quellen oder Dateieigenschaften habe ich deshalb mit passenden Werkzeugen und am aktuellen Code geprüft. Dabei zeigte sich, dass auch plausible und von mehreren Modellen wiederholte Aussagen falsch sein können. Für mich ist das ein wichtiger Beleg dafür, dass Verifikation vor Vertrauen stehen muss. Die Konsequenz ist nicht die Suche nach dem einen richtigen Modell, sondern eine konsequente Prüfung an Quelle und Code, unabhängig davon, welches Modell gerade antwortet.

3.5 KI Reflexion (Kristian Popov)

Mein Schwerpunkt lag während des Projekts auf den Modulen *Endliche Automaten* und *Reguläre Sprachen*. Ich habe dabei mehrere Modelle ausprobiert: Gemini 3.1 Pro, die kostenlose Version von ChatGPT sowie Claude Opus 4.8 und später Opus 5.

Jedes Modell hatte seine eigenen Stärken. Manche konnten theoretische Zusammenhänge besser erklären, andere waren stärker bei der Implementierung oder bei der Fehlersuche. Überwiegend habe ich mit Claude gearbeitet, weil das Modell bei Programmieraufgaben seltener fehlerhafte oder frei erfundene Vorschläge lieferte und ich die Antworten dadurch weniger oft nacharbeiten musste.

Erst mit der Zeit ist mir aufgefallen, dass die Modelle unterschiedlich viel Aufwand in die Erstellung einer Antwort investieren. Viele Modelle planen vor der eigentlichen Antwort mehr oder weniger ausführlich; der Planungsaufwand lässt sich meist in mehreren Stufen einstellen. Eine höhere Stufe bedeutet mehr verbrauchte Tokens, längere Wartezeit und höhere Kosten, führt aber nicht automatisch zu einem besseren Ergebnis. Für kleine, klar umrissene Änderungen war die niedrige Stufe ausreichend und zudem schneller. Erst wenn eine Aufgabe mehrere Dateien betraf oder eine Abwägung zwischen mehreren Lösungswegen verlangte, lieferte die höhere Stufe brauchbare Vorschläge. Deshalb habe ich je nach Aufgabe bewusst entschieden, welche Einstellung sinnvoll war.

3.5.1 KI als Co-Developer (Kristian Popov)

Die KI habe ich nicht über eine Chat-Oberfläche bedient, sondern direkt in die Entwicklungsumgebung eingebunden. Dadurch musste ich Code nicht mehr zwischen Editor und Chat kopieren. Stattdessen konnte das Modell auf die Projektdateien zugreifen, Änderungen vorschlagen und diese direkt im Projekt prüfen.

Besonders hilfreich war die KI bei wiederkehrenden Arbeiten: bei Refactorings und Optimierungen am Quellcode, beim Erzeugen von Boilerplate-Code, beim Schreiben von Unit-Tests sowie bei der Internationalisierung der Oberfläche. Gerade bei den Tests sparte sie viel Zeit. Für die Automatenlogik, das Parsen der Grammatiken und die Validierung der Programme konnte die KI aus der vorhandenen Implementierung schnell passende Testfälle ableiten, auch für Eingaben, an die ich selbst zunächst nicht gedacht hatte.

Außerdem half die KI beim Aufräumen des Projekts, etwa beim Beheben von TypeScript-Warnungen, beim Vereinheitlichen von Benennungen sowie beim Erstellen von Kommentaren und Methodendokumentationen. Mit den neueren Modellen und präzisen Prompts waren viele der vorgeschlagenen Änderungen bereits so gut, dass ich sie direkt übernehmen konnte. Trotzdem habe ich jede Änderung selbst überprüft und getestet.

Bei Architektur- und Implementierungsfragen habe ich die KI bewusst nicht selbst entscheiden lassen. Stattdessen ließ ich mir zunächst mehrere Lösungsansätze mit ihren Vor- und Nachteilen erklären und traf die Entscheidung anschließend selbst. Dieser Ansatz erforderte zwar etwas mehr Zeit, führte aber zu besser begründeten Architekturentscheidungen und verhinderte, dass ich Vorschläge der KI ungeprüft übernahm.

Möglich wurde diese Arbeitsweise durch mehrere Erweiterungen aus dem offiziellen Plugin-Verzeichnis `claude-plugins-official`. Für meine Module waren vor allem zwei davon relevant:

`typescript-lsp`
: bindet den Sprachserver von TypeScript über das Language Server Protocol (LSP) ein. Der KI stehen dadurch dieselben Typinformationen und Referenzen zur Verfügung wie der Entwicklungsumgebung. Bezeichner und Signaturen musste das Modell also nicht aus dem Quelltext erschließen, sondern konnte gezielt danach suchen, wodurch Änderungen über Dateigrenzen hinweg konsistent blieben.

`chrome-devtools-mcp`
: verbindet die KI über das Model Context Protocol (MCP) mit den Entwicklerwerkzeugen des Browsers. Konsolenausgaben und das Verhalten der laufenden Anwendung ließen sich damit direkt auswerten. Bei der Fehlersuche in den Simulatoren war das der größte praktische Gewinn: Ein fehlerhafter Zustandsübergang wurde am tatsächlichen Ablauf sichtbar und musste nicht aus dem Quelltext rekonstruiert werden.

Zusätzlich habe ich das Projekt einmalig mit Graphify [1] analysiert und daraus einen Wissensgraphen erzeugt. Graphify verbindet eine statische Analyse auf Basis von Tree-sitter mit einer semantischen Auswertung durch ein Sprachmodell. Die Knoten des Graphen stehen für Dateien, Klassen und Funktionen, die Kanten für deren Beziehungen untereinander. Zu Beginn einer Sitzung musste die KI das Projekt nicht erneut vollständig einlesen, sondern konnte den für die jeweilige Aufgabe relevanten Teilgraphen abfragen. Gerade bei Änderungen, die mehrere Dateien betrafen, blieb der notwendige Kontext dadurch besser erhalten.

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Für beide Module musste jede Übung mehr enthalten als nur die eigentliche Aufgabenstellung. Zusätzlich wurden eine Referenzlösung, ein Startprogramm sowie Beispielwörter benötigt, mit denen die Lösung automatisch überprüft werden konnte. Erst diese Kombination ergibt ein Lernangebot, das ohne Betreuung funktioniert.

Die KI habe ich hier vor allem zur Variantenbildung eingesetzt. Ausgehend von einer selbst entworfenen Referenzaufgabe ließen sich weitere Aufgaben desselben Typs erzeugen, mit anderem Alphabet, anderer Wortlänge oder anderem Schwierigkeitsgrad. Genau dafür war die KI gut geeignet: Die Struktur steht fest, variiert wird nur der Inhalt.

Ob die Aufgaben tatsächlich korrekt waren, habe ich aber nicht der KI überlassen, sondern im Simulator geprüft. Diese Prüfung läuft deterministisch ab: Geprüft werden alle Wörter über dem gemeinsamen Alphabet bis zur Länge vier, und für jedes dieser Wörter muss der Automat der Studierenden dieselbe Entscheidung treffen wie die Referenzlösung. Bewertet wird damit das tatsächliche Verhalten des Automaten und nicht die Einschätzung eines Sprachmodells. Aus demselben Grund habe ich jede generierte Aufgabe an der Simulation gegengeprüft, bevor sie in den Aufgabenkatalog aufgenommen wurde.

3.5.3 Kritische Reflexion (Kristian Popov)

Die Vorschläge der KI waren oft brauchbar, aber nicht immer richtig oder passend für die bestehende Projektstruktur. Wiederholt schlug die KI APIs vor, die in der verwendeten Version so nicht mehr existierten. Gelegentlich erfand sie auch Funktionen oder Bibliotheken, die es in dieser Form nie gegeben hatte. Gerade Sonderfälle wurden häufig übersehen, und einzelne Änderungen wirkten sich an Stellen aus, die im Vorschlag selbst gar nicht erwähnt waren. Besonders bei formalen Sprachen reicht es nicht aus, dass eine Lösung plausibel aussieht. Ob ein Automat die geforderte Sprache tatsächlich erkennt, zeigt sich erst beim Durchspielen konkreter Eingaben.

Ein weiteres Problem lag in der Struktur des Codes. Da sich *Endliche Automaten* und *Reguläre Sprachen* stark ähneln, duplizierte die KI bestehende Implementierungen, statt sie zu abstrahieren. So entstanden Simulator- und Übersichtsseiten, die sich zwischen den Modulen kaum unterschieden, und Funktionen, die nie in der Oberfläche eingebunden wurden und später wieder entfernt werden mussten. Betraf eine Änderung beide Module zugleich, etwa die einheitliche Gruppierung von Beispielen und Übungen nach Kategorien, reichte das Kontextfenster des Modells zudem nicht aus, um alle betroffenen Stellen zu erfassen. Solche Aufgaben musste ich in kleinere Schritte zerlegen.

Insgesamt hat mir die KI vor allem bei wiederkehrenden Aufgaben und beim Ausbau bereits vorhandener Funktionen viel Zeit gespart. Gerade bei theoretischen Konzepten und bei Änderungen an mehreren Modulen gleichzeitig war die erste Lösung aber selten schon die richtige. Den Wechsel zwischen den Modellen könnte künftig ein Orchestrator wie Sakana Fugu übernehmen, der je nach Aufgabe automatisch das passende Modell auswählt [2]. Das System erschien allerdings erst kurz vor Abschluss meiner Arbeit, sodass ich es nicht mehr einsetzen konnte. Für mich blieb die KI während des gesamten Projekts ein Werkzeug zur Unterstützung. Die fachlichen Entscheidungen und die Verantwortung für die Korrektheit der Implementierung lagen jedoch jederzeit bei mir.

4 Designentscheidungen der Fachmodule

4.1 Turing Maschine (Richard Schröder)

Die Turing-Maschine ist als eigenständiger Themenbereich innerhalb des Moduls „Grundlagen der Informatik“ umgesetzt. Der Bereich besteht nicht nur aus einem Simulator, sondern aus einem vollständigen Lern-Workflow mit Übersicht, Übungen, Beispielen und einer freien Simulation. Die Git-Historie zeigt dabei eine schrittweise Entwicklung: Zuerst wurden Datenmodell, Simulator-Schnittstelle und Debugger aufgebaut, danach folgten die editierbare Konfiguration, die responsive Banddarstellung, Beispiele und Übungen, Internationalisierung, Import/Export, automatische Speicherung und zuletzt weitere Tests sowie die Integration in gemeinsam genutzte Komponenten.

4.1.1 Entscheidungen in der Architektur (Richard Schröder)

- Die Implementierung trennt die Turing-Maschine konsequent in `domain`, `application`, `presentation` und `adapters`. Dadurch liegen fachliche Regeln, React-Zustand, UI-Komponenten und die Abbildung auf generische Tabellenkomponenten in getrennten Schichten.
- Das statische Maschinenprogramm und der Laufzeitzustand sind getrennte Modelle. `TMProgram` beschreibt Zustände, Alphabete, Blank-Symbol, Startzustand, Haltezustände,

Übergänge und Anfangsband. `TMRuntime` beschreibt aktuellen Zustand, Kopfposition, Bandinhalt, Schrittzähler und Halteinformation.

- Die eigentliche Simulation liegt in reinen Funktionen wie `createInitialRuntime`, `readCurrentSymbol` und `stepRuntime`. Diese Funktionen hängen nicht von React, i18n oder der Darstellung ab und sind deshalb gut isoliert testbar.
- Fehlende Übergänge werden fachlich als normales Halten modelliert und nicht als technischer Fehler. Ein fehlender Übergang erzeugt einen `haltReason` vom Typ `missing_transition`; ein regulärer Haltezustand erzeugt einen `haltReason` vom Typ `halt_state`. Dadurch kann die UI sauber zwischen einem erwartbaren Verhalten einer partiell definierten Turing-Maschine und echten Implementierungsfehlern unterscheiden.
- Die Validierung prüft Struktur und Konsistenz, aber nicht die Vollständigkeit der Übergangsfunktion. Es wird unter anderem geprüft, ob Start- und Haltezustände existieren, Symbole nur aus einzelnen Zeichen bestehen, das Blank-Symbol gültig ist und Übergänge auf existierende Zustände und Bandsymbole verweisen. Unvollständige Tabellen bleiben dagegen erlaubt, weil sie didaktisch und fachlich sinnvoll sind.
- Für die Simulation wird eine Klasse `TuringMachineSimulator` verwendet, die den gemeinsamen `ISimulator`-Vertrag der Anwendung erfüllt. Intern nutzt sie aber weiterhin die reinen Domain-Funktionen. Damit bleibt die Turing-Maschine kompatibel zur allgemeinen Simulationsengine, ohne die Fachlogik in Hooks oder UI-Komponenten zu verschieben.
- Der Simulator speichert eine Historie der Laufzeitzustände. Dadurch kann die UI neben „Schritt vorwärts“ auch „Schritt zurück“ anbieten. Die Historie liegt in der Simulator-Anwendungsschicht und bleibt von der Darstellung unabhängig.
- Die Editorlogik ist in `tmEditorReducer` und `useTMConfigEditor` ausgelagert. Der Reducer kümmert sich um Operationen wie Zustände hinzufügen, Zustände umbenennen, Bandsymbole umbenennen, Übergänge aktualisieren und Haltezustände verwalten. Wichtig ist dabei, dass Referenzen mitgeführt werden: Wird ein Zustand umbenannt, werden auch Startzustand, Haltezustände und Zielzustände in Übergängen angepasst.
- Die Übergangstabelle ist intern als verschachtelte Struktur aus Lesesymbol und Zustand modelliert. Für die Darstellung wird sie über `tmTableAdapter` in Header, Zeilen und aktive Tabellenzelle übersetzt. Diese Adapterentscheidung verhindert, dass die Domain-Schicht Tabellen- oder UI-Wissen enthält.
- Eingabealphabet und Bandalphabet werden nicht vollständig manuell gepflegt. Das Eingabealphabet wird aus dem Anfangsband abgeleitet; das Bandalphabet wird aus vorhandenem Bandalphabet, Anfangsband, Blank-Symbol, Lesesymbolen und Schreibsymbolen abgeleitet. Dadurch bleibt die Konfiguration für Lernende einfacher, während die Domain dennoch beide Konzepte kennt.
- Der Import und Export ist als eigener Transfer-Codec umgesetzt. Die aktuelle Architektur verwendet eine Bitstream-basierte, Base64-URL-sichere Kodierung mit Symbolregistern, Roh-UTF-8-Fallback und verschiedenen Übergangstabellen-Kodierungen. Die Historie zeigt, dass dies aus einem einfachen Base64-Proof-of-Concept zu einer kompakten, robusteren Speicherform weiterentwickelt wurde.
- Für gespeicherte Aufgabenstände wird der gemeinsame Training-Center-Speicher genutzt. Beispiele werden bewusst nicht dauerhaft überschrieben, während freie Simulationen und Übungen eigene Speicherkennungen verwenden. Dadurch bleiben eingebaute Beispiele reproduzierbar und Übungsfortschritte lokal erhalten.

- Die Übungsprüfung ist von der sichtbaren Simulation getrennt. Für jeden Validierungsfall wird ein Simulator mit dem jeweiligen Eingabewort gestartet, bis zum Halten oder bis zu einer Schrittgrenze ausgeführt und dann mit der erwarteten Ausgabe verglichen. Zusätzlich reicht eine passende Ausgabe nicht: Die Maschine muss in einem gültigen Haltezustand enden.

4.1.2 Entscheidungen im UI (Richard Schröder)

- Die Turing-Maschine nutzt den gemeinsamen Themenaufbau der Plattform: Kopfbereich mit Titel und Beschreibung, Tabs für Übersicht, Übungen, Beispiele und Simulation sowie die globale Sidebar mit Sprach- und Theme-Umschaltung.
- Die Simulation startet im Konfigurationsmodus. Lernende sehen zuerst Anfangsbandwort, Startzustand, Übergangstabelle, Haltezustände, Eingabealphabet, Bandalphabet sowie Import/Export. Erst nach dem Anwenden der Konfiguration wechselt die Ansicht in den Laufzeitmodus.
- Die Übergangstabelle ist der zentrale Editor. Zeilen stehen für Bandsymbole, Spalten für Nicht-Haltezustände. Haltezustände werden bewusst nicht als editierbare Übergangsspalten angezeigt, weil von ihnen aus keine weiteren Übergänge erwartet werden.
- Übergänge werden in der Form `(Next State, Write Symbol, Movement)` eingegeben. Das UI erlaubt leere Zellen und behandelt sie als nicht definierte Übergänge. Damit passt die Darstellung zur Architekturentscheidung, partielle Übergangsfunktionen zuzulassen.
- Fehler in einzelnen Tabellenzellen blockieren den Editor gezielt. Bei ungültigen Symbolen oder ungültigem Übergangsformat bleibt die betroffene Zelle editierbar, andere Änderungen und der Start der Simulation werden deaktiviert. So wird verhindert, dass ein halbvalider Tabellenzustand unbemerkt übernommen wird.
- Das Blank-Symbol wird sichtbar als ◇ dargestellt. Für die Eingabe ist zusätzlich die Tastaturform `<>` vorgesehen, die in das Blank-Symbol umgewandelt wird. Das reduziert Eingabehürden, ohne in der Darstellung ein uneindeutiges Leerzeichen verwenden zu müssen.
- Die Laufzeitansicht besteht aus Band, Steuerleiste, Statuskarte und Übergangstabelle. Das Band zeigt ein Fenster um die Kopfposition mit Indexbeschriftungen und einer Pfeilmarkierung für den aktuellen Kopf. Die Tabellenzelle des aktuellen Zustands und Symbols kann über den Adapter als aktive Zelle markiert werden.
- Das Band ist responsiv ausgelegt. Die Anzahl sichtbarer Bandzellen wird aus der verfügbaren Breite berechnet; Animationen werden nur für kleine Verschiebungen und sinnvolle Geschwindigkeiten ausgeführt. Dadurch bleibt die Darstellung auch bei kleinen Viewports und schnellen Simulationen beherrschbar.
- Die Steuerung verwendet die gemeinsamen Simulator-Controls der Plattform: Play, Pause, Schritt, Schritt zurück, Reset und Geschwindigkeitsregler. Damit verhält sich die Turing-Maschine ähnlich wie andere Simulatoren der Anwendung.
- Die Statuskarte zeigt aktuellen Zustand, aktuelles Symbol, Schrittzahl, Haltestatus und bei fehlenden Übergängen eine fachliche Erklärung. Bei der Live-Prüfung hielt die Maschine ohne definierten Übergang mit der Meldung, dass für den aktuellen Zustand und das aktuelle Symbol kein Übergang definiert ist.
- Übungen erweitern die Simulation um die Aktionen „Lösung laden“ und „Lösung prüfen“.

Das Prüfergebnis wird in einem Dialog als Tabelle mit Eingabe, Ausgabe, erwarteter Ausgabe, Bestehen und gültigem Haltezustand angezeigt.

- Die Oberfläche ist weitgehend lokalisiert. Eine sichtbare Auffälligkeit in der deutschen UI ist jedoch, dass in der Übergangstabelle der Zeilen-Hinzufügen-Text noch als „Add new row“ erscheint. Das deutet auf einen fehlenden oder nicht angebundenen Übersetzungseintrag in einer gemeinsam genutzten Tabellenkomponente hin.

4.1.3 Andere Entscheidungen (Richard Schröder)

- Die Beispiele und Übungen sind als Szenarien modelliert. Beispiele enthalten ein fertiges Programm; Übungen enthalten zusätzlich ein Starterprogramm, ein Lösungsprogramm, eine Speicher-ID und mehrere Validierungsfälle. Damit kann derselbe Simulator für freie Arbeit, Beispiele und bewertbare Übungen genutzt werden.
- Die enthaltenen Beispiele und Übungen decken verschiedene typische Aufgaben ab, etwa Bit-Invertierung, unäres und binäres Inkrementieren, Löschen des Bandes, Suchen von Mustern, Duplizieren von Nullen, Binäraddition und einfache Worterkennung. Die Übungen sind nach Schwierigkeitsgrad sortiert.
- Die Testsuite ist für diesen Bereich vergleichsweise breit. Es gibt Unit-Tests für Domain-Logik, Formatierung, Programmvalidierung, Transfer-Codec, Editor-Reducer, Tabellenadapter, Bandlogik, Lösungskontrolle und Simulator-Hook. Die E2E-Tests prüfen zusätzlich die sichtbare Bedienung, Beispiel- und Übungsnavigation, Local-Storage-Verhalten, Import/Export, Fehlervalidierung und das Erstellen einer Bit-Inverter-Maschine von Grund auf.
- Die Git-Historie legt nahe, dass die Architektur bewusst nachträglich stabilisiert wurde. Nach einem ersten Datenmodell und Simulator folgten größere Refactorings nach einer Architekturplanung, danach responsive Verbesserungen, Internationalisierung, Test-IDs, Übungsintegration und persistente Speicherung. Das spricht für eine iterative Entwicklung, bei der die UI zunächst funktionsfähig gemacht und anschließend stärker in Schichten getrennt wurde.
- Besonders sinnvoll ist die Entscheidung, „fehlender Übergang“ nicht als Programmfehler zu behandeln. Für Lernende ist eine unvollständige Turing-Maschine ein normaler Zwischenzustand beim Konstruieren. Gleichzeitig kann die Übungsprüfung streng bleiben, weil sie nur dann besteht, wenn die Maschine nicht nur die richtige Ausgabe produziert, sondern auch in einem vorgesehenen Haltezustand endet.
- Ein technischer Zielkonflikt liegt im Transferformat. Die kompakte Bitstream-Kodierung ist gut für kurze Exportstrings und lokale Speicherung, erschwert aber die manuelle Lesbarkeit. Diese Entscheidung ist plausibel, weil der Import/Export im UI als Kopier- und Austauschfunktion gedacht ist und nicht als primäres Bearbeitungsformat.
- Ein weiterer Zielkonflikt liegt in der abgeleiteten Alphabetpflege. Sie reduziert Eingabeaufwand und verhindert viele Inkonsistenzen, kann aber für fortgeschrittene theoretische Diskussionen weniger explizit sein als ein separat editierbares Eingabealphabet. Die aktuelle Lösung wirkt für die Lernplattform angemessen, weil sie den Editor einfacher hält.
- Aus Wartungssicht ist positiv, dass die meisten UI-Elemente stabile `data-testid`-Attribute besitzen. Das erleichtert E2E-Tests und war auch bei der Live-Inspektion hilfreich. Gleichzeitig sollten kleine Lokalisierungslücken in gemeinsam genutzten Komponenten weiter beobachtet werden, damit deutsche und englische Oberfläche konsistent bleiben.

4.2 Endliche Automaten (Kristian Popov)

Im Modul *Endliche Automaten* können Studierende deterministische und nichtdeterministische endliche Automaten selbst definieren und ein Eingabewort Schritt für Schritt ausführen. Jeder Schritt erscheint gleichzeitig in der Übergangstabelle und im Zustandsdiagramm. Die grafische Sicht macht dabei sichtbar, was aus der Tabelle allein nur mit Mühe hervorgeht: Zustände, Übergänge und den Weg, den das Wort durch den Automaten nimmt. Ergänzt wird die Einzelsimulation unter anderem durch einen Stapeltest für mehrere Wörter, die Umwandlung rechtslinearer Grammatiken in Automaten und Übungen mit automatischer Rückmeldung.

4.2.1 Entscheidungen in der Architektur (Kristian Popov)

Ursprünglich sollte das Modul nur deterministische endliche Automaten unterstützen. Die Unterstützung für nichtdeterministische Automaten kam erst später hinzu. Ein Symbol darf in mehrere Zustände zugleich führen, wodurch sich ein Automat oft unmittelbar aus der Beschreibung der Sprache ablesen lässt und zudem deutlich kompakter ausfällt. Der Schritt zum DFA, bei dem jede Mehrdeutigkeit aufgelöst werden muss, wird damit erst im Anschluss notwendig.

Für die Ausführung ergibt sich daraus eine einfache Konsequenz: Jeder DFA ist zugleich ein NFA. Zwei getrennte Ausführungspfade wären deshalb überflüssig gewesen. Der Kern arbeitet deshalb stets mit einer Menge aktiver Zustände; beim DFA enthält diese Menge höchstens ein Element. Der deklarierte Typ steuert nur die Validierung, nicht die Ausführung.

Der Simulator ist in fünf Verzeichnisse getrennt, die jeweils eine klar umrissene Aufgabe haben:

`domain`
: enthält die Fachlogik: den Datentyp des Automaten, die Schrittfunktion samt ε-Hülle, die Validierung, die Prüfung auf Determinismusverstöße, die Umwandlung aus einer Grammatik, den Stapellauf, die Auswertung der Übungen sowie die Berechnung des Diagrammlayouts.

`application`
: verbindet diese Logik mit React: ein Hook für den Ablauf der Simulation und ein Hook für den Editor, der die Eingaben über einen Reducer verwaltet.

`adapters`
: bereitet den Automaten für die Tabellendarstellung auf, also Kopfzeile, Zeilen je Symbol und die jeweils hervorzuhebende Zelle.

`presentation`
: enthält die Komponenten: Zustandsdiagramm, Übergangstabelle, Eingabewort, Statuskarte, Stapeltest und Grammatikeingabe.

`config`
: hält die Standardautomaten und die Beispiele.

Entscheidend an dieser Aufteilung ist, dass `domain` keine Abhängigkeit zu React besitzt. Die Automatenlogik bleibt dadurch ohne gerenderte Oberfläche testbar, und dieselben Funktionen konnten später im Modul *Reguläre Sprachen* unverändert wiederverwendet werden.

Ein Simulationsschritt verändert den bisherigen Laufzustand nicht, sondern erzeugt einen neuen. Dieser hält die Historie der aktiven Zustände und der genommenen Übergänge fest. Dadurch

sind sowohl ein Schritt zurück als auch der Sprung zu einem früheren Schritt möglich, ohne dass die Oberfläche eine zweite, eigene Historie führen muss.

Die Validierung liefert keine fertigen Texte, sondern Fehlercodes und die dazugehörigen Werte, etwa den betroffenen Zustand und das Symbol. Erst die Oberfläche übersetzt diese Informationen in Meldungen. So bleibt die Fachlogik von der Oberflächensprache unabhängig, und dieselben Fehler lassen sich konsistent mehrsprachig ausgeben.

Alle mitgelieferten Automaten sind partiell definiert: Fehlt für ein Symbol ein Übergang, wird das Wort abgelehnt. Auf einen zusätzlich gezeichneten Fangzustand, der nur noch sich selbst erreicht, wurde bewusst verzichtet. Er verändert die erkannte Sprache nicht, vergrößert aber das Diagramm und lenkt von der eigentlichen Struktur ab. Ein automatisierter Test stellt sicher, dass weder ein Beispiel noch eine Referenzlösung einen solchen Zustand enthält.

4.2.2 Entscheidungen im UI (Kristian Popov)

Ursprünglich war für die Simulation nur die Übergangstabelle zusammen mit dem Eingabewort vorgesehen. Formal genügt das: Die Tabelle definiert den Automaten vollständig, das Eingabewort zeigt die aktuelle Leseposition. Während der Entwicklung zeigte sich jedoch, dass sich der Ablauf so nur schwer verfolgen lässt. Welchen Weg ein Wort nimmt, ist aus der Tabelle nicht unmittelbar ersichtlich; Zyklen und Sackgassen werden erst sichtbar, wenn man mehrere Zeilen gedanklich miteinander verknüpft. Deshalb ergänzt ein Zustandsdiagramm die Tabelle.

Beide Sichten sind an denselben Simulationsschritt gebunden. Die Tabelle hebt genau die Zelle hervor, die dem im Diagramm markierten Übergang entspricht. Damit bleiben beide Darstellungen synchron: Die Tabelle liefert die formale Definition, das Diagramm die anschauliche Darstellung, und ihre Verbindung wird unmittelbar sichtbar.

Die Anordnung der Zustände wird aus der Größe des Automaten abgeleitet. Kleine Automaten werden auf einem Kreis angeordnet, größere über ein kräftebasiertes Verfahren verteilt, bei dem sich Zustände gegenseitig abstoßen und Übergänge sie zusammenziehen. Die Startpositionen werden aus dem Zustandsnamen berechnet und sind damit reproduzierbar. Ohne diese feste Streuung können symmetrische Automaten in einer ungeeigneten Lage zusammenfallen; zudem würde derselbe Automat bei jedem Aufruf anders aussehen. Das Diagramm lässt sich zoomen und verschieben, einzelne Zustände auch frei ziehen. Unterhalb einer Zoomstufe werden Beschriftungen ausgeblendet; aktive und fehlerhafte Übergänge behalten ihre Beschriftung jedoch immer, weil gerade sie in diesem Moment die relevante Information tragen.

Verstöße gegen den Determinismus werden nicht nur als Liste ausgegeben, sondern unmittelbar im Diagramm markiert. Damit ist die Ursache dort sichtbar, wo sie entsteht.

Der Editor umfasst alle Bestandteile der Definition eines Automaten: die Wahl zwischen DFA und NFA, das Eingabealphabet, genau einen Startzustand, beliebig viele Endzustände sowie das Eingabewort für den anschließenden Lauf. Die Zustandsmenge wird nicht als Text gepflegt, sondern direkt in der Übergangstabelle über Schaltflächen erweitert und verkleinert. Die Tabelle dient damit zugleich als Eingabemaske und als Anzeige. Alphabetsymbole sind auf jeweils ein Zeichen beschränkt, damit eine Eingabe wie `10` nicht versehentlich als ein einzelnes Symbol aufgenommen wird. Die Zeile für ε-Übergänge erscheint erst, wenn sie tatsächlich benötigt oder ausdrücklich angefordert wird; für einen DFA wäre sie nur eine Fehlerquelle.

4.2.3 Andere Entscheidungen (Kristian Popov)

Neben dem Einzelschritt gibt es einen Schrittverlauf, über den sich jeder bereits erreichte Zustand direkt auswählen lässt. Der gewünschte Schritt wird dabei vom Anfang des Laufs

an neu berechnet, statt ein separates Ergebnis zwischenzuspeichern. Bei den hier üblichen Wortlängen ist das unkritisch und stellt sicher, dass Anzeige und tatsächlicher Ablauf immer übereinstimmen.

Der Stapeltest ergänzt die schrittweise Simulation um den umgekehrten Blick. Statt ein einzelnes Wort genau zu verfolgen, werden mehrere Wörter auf einmal geprüft und mit ihrem Ergebnis aufgelistet. So lässt sich besser einschätzen, ob ein Automat die beabsichtigte Sprache erkennt und nicht nur das gerade ausprobierte Wort. Ein Klick auf ein Ergebnis übernimmt das Wort wieder in die Einzelsimulation, um den Ablauf gezielt nachzuvollziehen.

Ein Automat lässt sich außerdem aus einer rechtslinearen Grammatik erzeugen, also aus den Regeln, nach denen die Wörter der Sprache gebildet werden. Jedes Nichtterminal wird dabei zu einem Zustand, und eine Regel der Form `A -> aB` wird zum Übergang von `A` nach `B` beim Lesen von `a`. Regeln der Form `A -> a` führen in einen zusätzlichen akzeptierenden Zustand, Regeln, die auf das leere Wort abbilden, machen den jeweiligen Zustand selbst akzeptierend. Für die Eingabe werden zwei Schreibweisen akzeptiert, eine knappe mit einer Produktion pro Zeile und eine strukturierte mit Kopfzeile. Das Format wird automatisch erkannt und angezeigt, sodass zuvor keine Syntax ausgewählt werden muss. Die Aufteilung einer Alternative in Terminal und Nichtterminal erfolgt in zwei Durchläufen: Erst werden alle definierten Nichtterminale gesammelt, dann wird zerlegt. Das ist notwendig, weil Terminale und Nichtterminale denselben Namen tragen dürfen und sich `A -> AA` nicht aus den Zeichen allein auflösen lässt.

Beispiele und Übungen sind bewusst getrennt. Ein Beispiel ist ein fertiger Automat mit passendem Eingabewort und wird unverändert in die Simulation geladen; es dient dem Nachvollziehen eines vorhandenen Aufbaus. Eine Übung beginnt dagegen mit einem leeren Automaten und bringt Aufgabenstellung, Schwierigkeitsgrad, Referenzlösung und automatische Kontrolle mit.

Innerhalb beider Bereiche sind die Einträge danach gruppiert, was gegeben und was gesucht ist. Die Übungen zerfallen in zwei Kategorien: Bei *Sprache* → *Automat* ist die gesuchte Wortmenge umgangssprachlich beschrieben, bei *Grammatik* → *Automat* ist stattdessen eine rechtslineare Grammatik vorgegeben. Beide benutzen dieselbe Aufgabenmechanik und dieselbe Kontrolle; die zweite Kategorie trägt lediglich zusätzlich die Grammatik als Vorgabe. Die Beispiele sind entsprechend in deterministische Automaten, nichtdeterministische Automaten und die Umwandlung aus einer Grammatik unterteilt. Ohne diese Gliederung stünden Aufgaben mit unterschiedlicher Ausgangslage nebeneinander, und die Grammatikaufgaben wären allein an ihrem zusätzlichen Grammatikfeld zu erkennen.

Die Kontrolle der Übungen vergleicht nicht die Struktur zweier Automaten, sondern deren Verhalten. Sie prüft alle Wörter über dem gemeinsamen Alphabet bis zu einer festgelegten Länge; für jedes muss der Automat der Studierenden dieselbe Entscheidung treffen wie die Referenzlösung. Ein Strukturvergleich würde korrekte Lösungen ablehnen, die lediglich anders aufgebaut sind. Weicht eine Lösung ab, wird das erste unterscheidende Wort als Gegenbeispiel ausgegeben statt nur einer allgemeinen Fehlermeldung. Die begrenzte Wortmenge ist dabei bewusst eine didaktische Rückmeldung und kein formaler Äquivalenznachweis. Strukturelle Mängel wie ungültige Übergänge werden vorrangig gemeldet, da sie meist die eigentliche Ursache sind und eine Auflistung fehlgeschlagener Testwörter in diesem Fall in die Irre führen würde.

Der Bearbeitungsstand wird je Übung gespeichert und beim erneuten Öffnen wiederhergestellt. Ohne diese Trennung hätte das Wechseln zwischen zwei Übungen die jeweils andere Bearbeitung überschrieben.

4.3 Reguläre Sprachen (Kristian Popov)

Das Modul *Reguläre Sprachen* unterstützt die Arbeit mit regulären Sprachen und rechtslinearen Grammatiken. Es bietet drei miteinander verknüpfte Funktionen: Die Aufzählung der von einer Grammatik erzeugten Wörter, die Erzeugung einer passenden Grammatik aus einer Sprachbeschreibung sowie die Prüfung einzelner Wörter. Sprachbeschreibungen können entweder als Aufzählung oder in Mengenschreibweise eingegeben werden. Bei der Aufzählung einer Sprache werden die Wörter nach ihrer Länge sortiert. Die Ausgabe ist dabei auf eine maximale Wortlänge und Wortanzahl begrenzt. Bei der Wortprüfung zeigt das Modul neben der Annahme oder Ablehnung auch den Grund für ein abweichendes Ergebnis an, beispielsweise einen nicht möglichen Übergang. Beide Umwandlungsrichtungen sind zusätzlich als Übungen mit automatischer Rückmeldung verfügbar.

4.3.1 Entscheidungen in der Architektur (Kristian Popov)

Der Simulator besteht aus einer koordinierenden Komponente und fachlich getrennten Bausteinen. Die Komponente `RLSimulator` verwaltet die drei Arbeitsmodi, den Wechsel zwischen ihnen und den Einstieg in Übungen. Sie enthält selbst keine formale Logik. Diese liegt vollständig im Verzeichnis `domain` und ist damit unabhängig von React testbar.

Die fachlichen Bausteine sind wie folgt aufgeteilt:

Grammatikverarbeitung
: Der Grammatikparser und die Validierung lesen die Eingabe ein, prüfen die Struktur einer rechtslinearen Grammatik und überführen sie in gemeinsame Datentypen.

Sprachbeschreibung
: Ein eigener Parser verarbeitet Aufzählungen und Mengenschreibweisen. Er liefert eine strukturierte Beschreibung aus Faktoren, Exponenten und Nebenbedingungen, die anschließend in eine Grammatik umgewandelt werden kann.

Umwandlung und Vereinfachung
: Die Umwandlung erzeugt aus einer Sprachbeschreibung eine strikt rechtslineare Grammatik. Ergänzende Funktionen fassen gleichwertige Nichtterminale zusammen und formatieren das Ergebnis für die Ausgabe.

Sprachauswertung
: Die Aufzählung erzeugt Wörter bis zu den festgelegten Grenzen; die Worterkennung prüft ein einzelnes Wort gegen eine Grammatik. Die Übungsprüfung baut auf denselben Funktionen auf und ergänzt sie um die Auswertung der vorgegebenen Testfälle.

Wiederverwendet werden der Grammatikparser, die Grammatikvalidierung und die gemeinsamen Datentypen. Eine eigene Ableitungslogik direkt auf den Produktionen hätte zwei Implementierungen erfordert, die dasselbe Verhalten abbilden müssen. Mit der gemeinsamen Grundlage stimmen Aufzählung, Worterkennung und Übungsprüfung stets überein.

Für die Sprachbeschreibung habe ich einen eigenen Parser geschrieben, der zwei Formen versteht:

Aufzählung
: listet die Wörter unmittelbar auf:
`L = {"010", "0010"}`

Mengenschreibweise
: beschreibt ein Muster mit Variablen und Nebenbedingungen:

```
L = {a^n c b^m | n, m > 0}
```

Endet eine Aufzählung mit einer Fortsetzungsmarkierung, versucht der Parser, daraus ein wiederkehrendes Muster abzuleiten, statt die Wörter nur wörtlich zu übernehmen. So lässt sich eine unendliche Sprache angeben, ohne die Mengenschreibweise schon beherrschen zu müssen.

Bestimmte Eingaben werden bewusst abgelehnt, auch wenn sie sich formal beschreiben lassen. Verwendet dieselbe Variable mehrere Faktoren, wie in `a^n b^n`, koppelt das deren Längen aneinander. Der Musterfall `a^n b^n` ist nicht regulär; die unterstützte Sprachbeschreibung lässt solche gekoppelten Längenbedingungen deshalb generell nicht zu und weist sie mit einer eigenen Meldung zurück. Obere Schranken werden ebenfalls nicht unterstützt, obwohl sie endliche und damit reguläre Sprachen beschreiben. Sie würden die Eingabesprache unnötig erweitern und lassen sich klarer als Aufzählung angeben. Diese Fälle ausdrücklich zu erklären, ist lehrreicher, als sie stillschweigend zu verwerfen.

Die erzeugte Grammatik ist strikt rechtslinear, es entstehen also ausschließlich Regeln der Form `A -> aB` und `A -> a` sowie die Regel $A \to \varepsilon$. Eine frühere Fassung gab eine kompaktere Form wie `S -> A c B` aus. Diese Schreibweise ist zwar kürzer, aber eine allgemeine kontextfreie Regel und verlässt damit die Typ-3-Grammatiken, um die es in diesem Modul gerade geht.

Um trotzdem eine lesbare Ausgabe zu bieten, gibt es zwei Sichten auf dasselbe Ergebnis statt zweier Formalismen: Die detaillierte Sicht zeigt ein Nichtterminal je Präfix, die vereinfachte Sicht führt Nichtterminale zusammen, die dieselben Wörter ableiten. Bei der vereinfachten Sicht werden äquivalente Teilbäume zusammengeführt. Sie bleibt rechtslinear und kann die Grammatik nur verkleinern, nie ihre Sprache verändern.

Alle Stellen, die eine rechtslineare Grammatik ausgeben, benutzen dieselbe Namensvergabe und Ausgabefunktion. Zuvor lag diese Logik in drei leicht unterschiedlichen Kopien vor. In einer davon verbrauchte ein weggelassener Endzustand einen Buchstaben, sodass in der Ausgabe eine Lücke entstand und etwa `E` übersprungen wurde. Eine gemeinsame Funktion schließt solche Abweichungen aus und stellt sicher, dass eine exportierte Grammatik nach dem Import dieselben Namen trägt.

4.3.2 Entscheidungen im UI (Kristian Popov)

Die drei Arbeitsweisen sind über eine Modulauswahl erreichbar. Ihre Komponenten bleiben im Hintergrund erhalten und werden nur ein- oder ausgeblendet. Dadurch bleiben Eingaben beim Wechsel zwischen den Modi bestehen. Würden die Modi jeweils neu aufgebaut, ginge die begonnene Arbeit verloren und die Ansicht spränge auf ihr Standardbeispiel zurück.

Ist eine Übung geöffnet, ersetzt eine geführte Arbeitsfläche den freien Umwandler. Vorgabe und Aufgabe stehen dabei nebeneinander: In der einen Richtung ist die Grammatik gegeben und die Sprache gesucht, in der anderen umgekehrt. Ein Moduswechsel während einer offenen Übung würde den aktuellen Bearbeitungsstand verwerfen. Deshalb wird vorher eine Bestätigung verlangt.

Fehlermeldungen erscheinen unmittelbar unter dem jeweiligen Eingabefeld, aber erst, nachdem überhaupt etwas eingegeben wurde. Ein leeres Feld ist kein Fehler, sondern der Ausgangszustand. Zusätzlich lässt sich die unterstützte Syntax direkt neben dem Eingabefeld aufklappen, damit die Schreibweise nicht erst gesucht werden muss.

4.3.3 Andere Entscheidungen (Kristian Popov)

Die Kontrolle der Übungen vergleicht keine Texte. Für die Richtung *Sprache→ Grammatik* sind zu jeder Aufgabe Wörter hinterlegt, die erzeugt werden müssen, und Wörter, die nicht erzeugt werden dürfen. Geprüft wird, ob die eingegebene Grammatik alle geforderten Wörter ableitet und alle ausgeschlossenen ablehnt. Ein Textvergleich mit einer Musterlösung wäre hier unbrauchbar, da dieselbe Sprache von vielen unterschiedlichen Grammatiken erzeugt werden kann und die Sprachen zum Teil unendlich sind. Die Rückmeldung nennt daher falsch erzeugte und fehlende Wörter statt nur einer Ja-Nein-Aussage. Da nur vorbereitete Beispiele geprüft werden, dient auch diese Kontrolle der gezielten Rückmeldung und ist kein allgemeiner Beweis für die Gleichheit zweier Sprachen.

Für die Richtung *Grammatik → Sprache* verwendet die Kontrolle dieselben Grenzen für Wortlänge und Wortanzahl wie die Schaltfläche zum Anzeigen der Lösung. Andernfalls könnte eine angezeigte Lösung die anschließende Prüfung nicht bestehen.

Beim Import und Export unterscheiden sich die beiden Formate bewusst. Eine Grammatik wird strukturiert kodiert, also in Nichtterminale, Terminale und Produktionen zerlegt und bitweise gepackt. Eine Sprachbeschreibung wird dagegen als Text übernommen, weil sie vom Benutzer selbst formuliert ist und ihre Schreibweise erhalten bleiben soll. Beide Seiten prüfen beim Kodieren und beim Dekodieren, dass der Inhalt gültig ist.

Jeder Modus speichert seinen Stand in einem eigenen Bereich der lokalen Ablage. Ohne diese Trennung hätten sich die Grammatik des Aufzählungsmodus und die des Worterkenners gegenseitig überschrieben, da beide auf demselben Eingabetyp arbeiten.

Die Aufgabenstellungen selbst sind in einer sprachneutralen Notation hinterlegt und nicht in den Übersetzungsdateien. Eine Produktion wie `START = "1", A` ist in beiden Oberflächensprachen identisch; eine Übersetzung würde hier nur zusätzliche Fehlerquellen schaffen.

4.4 Zahlensysteme (Enrico Ebert)

4.4.1 Entscheidungen in der Architektur (Enrico Ebert)

4.4.2 Entscheidungen im UI (Enrico Ebert)

4.4.3 Andere Entscheidungen (Enrico Ebert)

Die Punkte sind nur Vorschläge ...

4.5 Moore & Mealy Automaten (Markus Nöth)

4.5.1 Entscheidungen in der Architektur (Markus Nöth)

4.5.2 Entscheidungen im UI (Markus Nöth)

4.5.3 Andere Entscheidungen (Markus Nöth)

Die Punkte sind nur Vorschläge ...

4.6 Disjunktive Normalform (Duy Tan Pham)

Die „Disjunktive Normalform“ (DNF) ist ein eigener Themenbereich im Modul „Grundlagen der Informatik“. Sie folgt dem gleichen Aufbau wie die anderen Themen: Übersicht, Übungen, Beispiele und eine freie Simulation. Die Inhalte bauen schrittweise aufeinander auf: von der kanonischen DNF über die minimale DNF bis zur optional global kleinsten Formel. Die einzelnen

Schritte der Ableitung werden dabei sichtbar gemacht. Die Git-Historie zeigt die Entwicklung Schritt für Schritt: zuerst das Datenmodell und die Ableitung aus der Wahrheitstabelle, dann der zweite Aufgabenmodus (Wahrheitstabelle nach DNF), danach die Übungs- und Feedbacklogik, die Vereinfachung mit Lösungsweg und zuletzt die Umstellung der Minimierung von einer Greedy-Auswahl auf ein exaktes Verfahren.

Die inhaltliche Ausrichtung folgt den vorhandenen Übungsunterlagen der Lehrveranstaltung. Dort wird die DNF aus Wahrheitstabellen mit drei und vier Variablen gebildet. Der Modus „Wahrheitstabelle nach DNF“ setzt genau diesen Aufgabentyp interaktiv um. Die Grenze von vier Variablen deckt damit zugleich die vorgesehenen Übungsbeispiele ab.

4.6.1 Entscheidungen in der Architektur (Duy Tan Pham)

- Der Bereich ist klar in `domain`, `application`, `adapters` und `presentation` getrennt. Die gesamte Fachlogik liegt in `domain`: Parser, Wahrheitstabelle, DNF-Ableitung, Minimierung und Prüfung. Diese Funktionen brauchen kein Framework. Sie sind unabhängig von React und lassen sich dadurch einzeln testen.
- Der boolesche Ausdruck wird von einem eigenen Parser (`dnfParser.ts`) in einen Syntaxbaum umgewandelt. Aus diesem Baum wird die Wahrheitstabelle erzeugt. Die Wahrheitstabelle ist die gemeinsame Grundlage für alle weiteren Schritte. Der Parser erlaubt bewusst mehrere Schreibweisen (`!`, `&`, `|` und ebenso ¬, ∧, ∨) und meldet Fehler mit Position.
- Die Ableitung trennt drei Ergebnisstufen sauber: die kanonische DNF (`dnfDerivation.ts`, ein Vollminterm je Eins-Zeile), die minimale DNF (`quineMcCluskey.ts`) und die optional global kleinste Formel (`minimalExpression.ts`). Darstellung und Rechenform sind getrennt. So ist die Anzeige nicht an die interne Bitdarstellung der Implikanten gebunden.
- Die wichtigste fachliche Entscheidung betrifft die Minimierung. Die erste Version wählte die restliche Überdeckung nach Quine-McCluskey *greedy* aus. Dadurch konnte sie die beste Lösung verpassen. Diese Auswahl wurde durch ein exaktes Branch-and-Bound über den zyklischen Kern ersetzt (Funktion `selectCover`). Das Verfahren liefert dieselbe Optimalitätsgarantie wie das Petrick-Verfahren und findet eine DNF mit minimaler Literalzahl. Durch das Abschneiden von Zweigen (Pruning) verwirft es Teile des Suchraums, die keine bessere Lösung mehr liefern können, ohne den Petrick-Ausdruck vollständig auszumultiplizieren.
- Für die optional kleinste Formel wird zusätzlich eine exakte Synthese über den Funktionsraum berechnet. Für Ausdrücke aus Literalen, die mit UND und ODER verbunden werden, liefert sie bis vier Variablen das globale Literalminimum. Negationen stehen dabei nur direkt an Variablen. Ab fünf Variablen wird der Zustandsraum (2^{2^n}) praktisch zu groß. Deshalb ist das Werkzeug bewusst auf vier Variablen begrenzt.
- Die Vier-Variablen-Grenze ist an drei Stellen abgesichert: in der Prüfung der Anwendungsschicht, in der Tabellen-Eingabe der Oberfläche und im Import. So bleibt jede Aufgabe in dem Bereich, in dem die exakte Synthese sicher funktioniert.
- Die Prüfung ist von der Darstellung getrennt. Ein eingegebener Ausdruck wird für jede Zeile der Wahrheitstabelle ausgewertet und nicht durch einen reinen Textvergleich bewertet. Dadurch werden logisch gleichwertige Schreibweisen erkannt. Die Übungsprüfung (`dnfExerciseCheck.ts`) unterscheidet zusätzlich zwischen „korrekt“, „äquivalent, aber nicht kanonisch“ und „nicht äquivalent“. Sie benennt außerdem fehlende oder überzählige Terme. Die Vereinfachungsprüfung (`dnfSimplificationCheck.ts`) prüft zuerst die logische Gleichwertigkeit und anschließend, ob die Literalzahl das exakte Minimum erreicht.

- Import und Export nutzen wie die anderen Simulatoren einen kompakten, URL-sicheren Base64-Code und sind an den gemeinsamen Training-Center-Speicher angebunden. So verhält sich der DNF-Bereich beim Speichern und Teilen genauso wie der Rest der Plattform.

4.6.2 Entscheidungen im UI (Duy Tan Pham)

- Der DNF-Bereich bietet zwei Aufgabenmodi: „Ausdruck nach Wahrheitstabelle“ und den umgekehrten Weg „Wahrheitstabelle nach DNF“. Beim zweiten Modus stellen die Lernenden aus einer gegebenen Tabelle die DNF selbst auf. Beide Übungen nutzen dieselben Bausteine.
- Die einzelnen Schritte der Ableitung werden sichtbar gemacht: von den Eins-Zeilen der Tabelle über die passenden kanonischen Terme bis zur minimalen Form. Bei der Vereinfachung wird nicht nur das Ergebnis gezeigt, sondern auch der Lösungsweg. So ist es für die Lernenden leichter nachzuvollziehen.
- Die Beispiele zeigen einen vollständig vorbereiteten Lösungsweg und dienen dem schrittweisen Nachvollziehen. In den Übungen bearbeiten die Lernenden dagegen Wahrheitstabellen oder DNF-Ausdrücke selbst. Die Rückmeldung wird dabei aus der DNF-Fachlogik abgeleitet.
- Kleinere Wahrheitstabellen werden vollständig angezeigt. Bei `16` Zeilen, also bei vier Variablen, gibt es eine kompakte, scrollbare Ansicht, die sich auf eine Vollansicht umschalten lässt. Dadurch bleibt die maximale Darstellungshöhe begrenzt.
- Folgeaktionen bleiben sichtbar und werden ausgegraut, bis ihre Voraussetzungen erfüllt sind, statt plötzlich aufzutauchen. Rein kontextabhängige Schaltflächen, etwa für die Vollansicht einer großen Tabelle, erscheinen dagegen nur dann, wenn sie tatsächlich benötigt werden. Dadurch bleibt die Bedienung vorhersehbar.
- Eingaben werden vereinheitlicht: Lernende dürfen `!`, `&` und `|` tippen, angezeigt wird aber immer $\neg$, $\wedge$, $\vee$. Das macht die Eingabe leichter, ohne die Darstellung uneinheitlich zu machen.
- Für die Fehlersuche stehen Import und Export zur Verfügung. So können Lernende einen Aufgabenstand teilen und nachvollziehen. Der DNF-Bereich nutzt außerdem den gemeinsamen Themenaufbau mit Kopfbereich, Tabs und globaler Sidebar, damit er sich wie ein Teil derselben Lernplattform anfühlt.

4.6.3 Andere Entscheidungen (Duy Tan Pham)

Die kanonische und die minimale DNF werden bewusst getrennt dargestellt. In der kanonischen DNF enthält jeder Minterm alle Variablen. Eine minimale DNF kann dagegen kürzere Terme besitzen, etwa $(a \wedge c) \vee (b \wedge c)$. Die getrennte Darstellung macht diesen fachlichen Unterschied nachvollziehbar und verhindert, dass eine korrekte Vereinfachung fälschlich als unvollständig angesehen wird.

4.7 Encoding (Enrico Ebert)

4.7.1 Entscheidungen in der Architektur (Enrico Ebert)

4.7.2 Entscheidungen im UI (Enrico Ebert)

4.7.3 Andere Entscheidungen (Enrico Ebert)

Die Punkte sind nur Vorschläge ...

4.8 Java (Markus Nöth)

4.8.1 Entscheidungen in der Architektur (Markus Nöth)

4.8.2 Entscheidungen im UI (Markus Nöth)

4.8.3 Andere Entscheidungen (Markus Nöth)

Die Punkte sind nur Vorschläge ...

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Enrico Ebert)

CI/CD-Pipeline (GitLab Runner auf Raspberry Pi), automatisiertes Linting, Formatierung (Prettier), Unit-Tests (Vitest) und E2E-Tests (Playwright)

5.2 Abgleich mit dem Lastenheft (Kristian Popov)

Wurden die Qualitätsziele (Performance, Usability, Barrierefreiheit durch Dark Mode und i18n) erreicht?

5.3 Lessons Learned (Alle)

Herausforderungen in der Team-Kollaboration (Git-Workflow, Merge-Requests, geteilte Architektur) und technische Hürden

5.4 Design-Iterationen (Duy Tan Pham)

Welche Entscheidungen mussten im Prozess revidiert werden?

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Alle)

Sieben interaktive Fachmodule in zwei Lernbereichen

6.2 Perspektiven (Kristian Popov)

Gamification (Badges/Punkte), LMS-Anbindung (Moodle/ILIAS) und Mobile Learning (responsive Optimierung)

Anhang

Abkürzungsverzeichnis

LSP Language Server Protocol; Protokoll zwischen Editor und Sprachserver für Codenavigation, Typinformationen und Refactoring

MCP Model Context Protocol; Offener Standard, über den Sprachmodelle externe Werkzeuge und Datenquellen anbinden

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [3]

Literatur

[1] S. Shamsi. „Graphify - Multi-modal Knowledge Graph Builder," besucht am 31. Juli 2026. Adresse: `https://graphify.net/`.

[2] Sakana AI. „Sakana Fugu - Multi-agent System as A Model," besucht am 31. Juli 2026. Adresse: `https://sakana.ai/fugu/`.

[3] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt," besucht am 14. Juni 2026. Adresse: `https://thws.de`.